

Predicting Credit Card Default Using Machine Learning

Semester Project to the Lecture
Supervised Machine Learning
Fall 2025

Nikolas Kehler

Robert Puselja

Masters in Economics
University of Lucerne

Supervised by

Dr. Massimo Mannino

Abstract

Credit card default prediction is a binary classification problem with a usually strong class imbalanced and heavy asymmetric economic costs – with not catching a future default resulting in typically much higher costs than classifying a healthy customer wrongly. While most applications optimize for ROC-AUC, such metrics ignore the cost structure of the relevant financial institutions, leading us to incorporate a hypothetical cost model and choosing the best models not only by ROC-AUC but by threshold-optimization and the resulting costs. Although several models achieve similar ROC-AUC values on the test set, their expected costs differ once the cost-optimal thresholds are applied, with particularly the weighted Gradient Boosting Model achieving the lowest expected costs. Our results demonstrate that AUC-maximization alone is insufficient for decision-making in credit risk prediction, with cost-minimization being the necessary addition to making such models truly optimized for financial institutions.

Table of Content

1. Introduction	1
2. Dataset	1
2.1 Dataset Description	1
2.2 Data Cleaning and Preparation Steps	2
2.3 Feature Engineering	3
2.4 Dataset Splitting	4
3. Modelling Framework	5
3.1 Model Training and Evaluation Strategy	5
3.2 Handling Class Imbalance	6
3.3 Cost Structure and Threshold Optimization	6
3.3.1 Missed Default Costs	7
3.3.2 Opportunity Cost of Rejecting a Non-Defaulting Customer	7
4. Results	8
4.1 Overview	8
4.1.1 Linear Models	8
4.1.2 Nonlinear Models	9
4.1.3 Business Cost Comparison	10
4.2 AUC vs. Business Cost	10
4.3 Threshold Behaviour and Generalization	12
4.4 Weighted vs. Unweighted	15
4.5 Variable Importance	17
5. Model Selection and Deployment Simulation	18
6. Discussion	19
7. Conclusion	19
Appendix A: Model Specifications and Hyperparameter Tuning	20

1. Introduction

Credit card default prediction is a critical task for financial institutions to manage credit risk. Accurately identifying customers who are at risk of defaulting on their credit card payments enables banks to take proactive measures and mitigate potential losses. Traditional statistical models have been used for credit scoring, but recent advances in **machine learning (ML)** offer the potential for improved predictive performance by capturing complex nonlinear patterns in customer data. In this project, we apply a range of supervised ML algorithms to predict credit card default and compare their performance.

Importantly, credit scoring decisions are inherently cost-sensitive. Misclassification errors are asymmetric: approving a future defaulter results in direct financial losses, while rejecting a creditworthy customer lead to opportunity costs in the form of foregone future profits. Consequently, purely statistical performance metrics such as accuracy or AUC are insufficient to fully evaluate model quality from a business perspective. This study therefore explicitly incorporates economic cost considerations into both model evaluation and decision threshold selection.

The objective of this study is to build predictive models using various ML techniques and evaluate which model performs the best in forecasting credit card payment defaults, that is which one minimizes costs while providing stable results. We use a real-world dataset of credit card clients to train and validate **six different models** (logistic regression, Lasso and Ridge regularized regressions, decision tree, random forest and gradient boosting machine). The following report details the dataset and preprocessing steps, exploratory data analysis results, modelling approach for each algorithm including cross-validation and hyperparameter tuning, a comparative evaluation of model performance, a simulation deployment of the best model and conclusions on key findings and implications.

2. Dataset

2.1 Dataset Description

For this analysis we use the Default of Credit Card Clients dataset, originally collected in Taiwan and widely used in default prediction research (Yeh & Lien, 2009). The dataset contains information on 30,000 credit card clients, including their demographic background, credit account details, recent payment history and whether they defaulted on their payment the following month. The binary outcome variable is default (1 indicating the client defaulted on payment, 0 indicating no default). In our sample, approximately 22% of the clients defaulted, while 78% did not.

Each client is described by 23 explanatory variables, which can be grouped as follows:

- **Credit Balance:** The amount of given credit limit in New Taiwan dollars, including any supplementary cards. This reflects the customer's credit line, which may be related to their repayment behaviour. The category contains this variable: **X1**
- **Demographics:** ID contains the ID of the customer, Sex (gender: 1 = male, 2 = female), Education (1 = graduate school, 2 = university, 3 = high school, 4 = others), Marriage (1 =

married, 2 = single, 3 = others) and Age (age in years). These variables provide background information of the customer. The category contains these variables: **ID, X2, X3, X4, X5**

- **Payment History:** There are six variables representing the repayment status in April, May, June, July, August, and September 2005 respectively. These are integer codes indicating whether the client paid their previous monthly dues on time or was over time. A value of `-1` indicates the account was paid in full and on time, 1 means payment was 1 month late, 2 means 2 months late, and so on up to 9, which indicates 9 or more months of delay. The category contains these variables: **X6, X7, X8, X9, X10, X11**.
- **Bill Statements:** The amount of the bill statement for each month from April to September 2005. These six continuous variables record the client's total outstanding balance at the end of each month. The category contains these variables: **X12, X13, X14, X15, X16**.
- **Payments:** The amount of payment made in each month from April to September 2005. These six features indicate how much the client paid in each of the past six months toward their bill. The category contains these variables: **X18, X19, X20, X21, X22, X23, Y**.

To conclude, the dataset provides a comprehensive recent financial history for each client, along with basic demographic information. Such data is suitable for training ML models to predict the October 2005 payment default.

2.2 Data Cleaning and Preparation Steps

Before modelling, we performed data cleaning to ensure the dataset was properly formatted for analysis.

1) Removal of non-informative identifiers

The raw data contained an ID column that served only as a row index and did not provide any predictive information. We therefore removed this variable.

2) Systematic sanity checks

Before modifying any values, we verified the structure and quality of the dataset:

- We inspected the first and last rows, variable types, and summary statistics.
- We checked for missing values (NAs) across all variables. None were found.
- We checked for duplicated rows and no duplicates were detected.
- We inspected numeric variables for impossible values and extreme outliers. While extreme values exist we retained them because they represent real high-risk credit behaviors and contain relevant signal rather than noise.
- We performed programmatic validity checks for categorical variables to identify values outside the expected domain (e.g., Education cannot be 0 or 6). This confirmed that several variables contained out-of-range codes.

3) Recoding inconsistent categorical variables

Several categorical features were encoded as integers with unclear meaning. Based on documentation and the invalid-value checks, we converted them into interpretable factor variables and consolidated rare or undefined categories:

- **Sex:** Originally encoded as 1 and 2. We recoded these into *Male* and *Female*.

- **Education:** Theoretical values are 1–4, but the dataset contained additional codes {0, 5, 6}. Following standard practice in similar studies, we grouped these invalid codes into a single *Unknown/Other* category.
- **Marriage:** The valid categories are 1 = Married, 2 = Single. Codes 0 and 3 occur only rarely and likely indicate missing or unspecified information, so we grouped them into an *Other/Unknown* marital-status category.

4) **Verification after cleaning**

After recoding, we checked the dataset to ensure that:

- All categorical values were within their defined sets.
- No new NAs were generated during recoding.
- All numeric variables were correctly stored in numeric format.
- The target variable (*DEFAULT*) was converted to a factor with levels *No* and *Yes* so that downstream classification models (caret) interpret it correctly.

These preprocessing steps ensured that the dataset was fully consistent before feature engineering and modeling. No observations were removed during cleaning, because there were no missing values, and financial “outliers” are informative rather than erroneous.

2.3 Feature Engineering

After completing data cleansing, we performed feature engineering to better capture relevant repayment behaviour and reduce redundancies in the dataset. During the analysis, we found strong correlations between monthly bill amounts and monthly payment variables. Instead of relying on these highly correlated values, we created additional features that summarise repayment behaviour in a more meaningful and interpretable way. The goal was to highlight patterns that are economically relevant to credit default risk while keeping the feature set manageable.

First, we constructed payment to bill ratios for each of the six observed months. These ratios compare the amount paid in a given month to the corresponding bill amount. This feature directly reflects how much of the outstanding balance was actually repaid. A value close to one indicates that the bill was paid in full, while lower values indicate partial repayment. This relative measure is more informative than absolute payment amounts alone, since it distinguishes between customers who pay similar amounts but face very different bill sizes. Persistently low payment to bill ratios can signal financial stress or repayment difficulties and are therefore highly relevant for predicting default.

In addition, we created aggregate features to summarize customer behavior over the full six month period. *TotalBill* represents the sum of all bill statements, while *TotalPay* represents the sum of all payments made during the same time span. These variables provide an overall view of the customer’s credit exposure and repayment activity. *TotalBill* reflects how much debt the customer accumulated, whereas *TotalPay* captures the overall ability or willingness to repay. Using these aggregate measures allows the models to consider longer term behavior instead of relying only on month specific information, while also reducing redundancy among highly correlated monthly variables.

Another important engineered feature is *MaxDelay*. This variable captures the maximum repayment delay observed across the six months. It represents the worst delinquency a customer

experienced during the observation period. Even a single severe delay can strongly increase the risk of default, which makes this feature particularly informative. By summarizing repayment history into one variable, MaxDelay allows the models to easily identify customers with a history of serious payment problems.

To account for changes over time, we also included a BillTrend feature. This variable describes how the bill amounts developed across the six months. It indicates whether the customer's outstanding balance increased or decreased over time. An increasing bill trend suggests that the customer is accumulating debt, while a decreasing trend indicates that balances are being paid down. Capturing this dynamic helps the models identify customers whose financial situation may be deteriorating, which is an important aspect of default risk.

We measured the variation in monthly payments using a payment variability feature. This variable reflects how stable or unstable the payment amounts were over time. Large variation in payments may indicate irregular income or inconsistent repayment behavior, both of which can be associated with higher default risk. In contrast, stable payment patterns may signal more predictable and manageable financial behavior.

Regarding preprocessing, feature scaling was applied selectively. For models that are sensitive to variable scale, such as regression models and support vector machines, all continuous numeric features were standardized using z score normalization. Tree based models were trained on unscaled data, since their splitting rules are not affected by feature magnitude. By combining carefully selected engineered features with model appropriate preprocessing, we ensured that the dataset captures key repayment patterns while remaining interpretable and suitable for different modeling approaches.

2.4 Dataset Splitting

We partitioned the data in two stages using stratified sampling in order to preserve the same class imbalance between defaults and non-defaults. In the first stage, the full dataset was divided into a **training pool** (80 %) and a **hold-out “real” dataset** (20%), with the training pool being used for training and testing, while the Hold-out dataset was only used as a final stability check and a pseudo-deployment simulation. In the second stage, the training pool was further split into the **training set** (75% of the pool, i.e. 60% of total data) and the **test set** (25% of the pool, i.e. 20% of total data).

Model selection in training stage, hyperparameter tuning and threshold optimization were performed only on the training set, with the test set being used for out-of-sample evaluation and selecting the final best model.

Since having three models sensitive to feature scale (three types of regression models), the numeric predictors were standardized using z-score normalization, while the other models were trained on unscaled data.

Additionally, with our dataset containing 30'000 observations in total and training time exceeding two hours for some models, we made use of a stratified 10% subset of the training data (10% of total data) for initial grid search, followed by refined fit on the full train data set. This ensured that tuning was timewise feasible for this project.

3. Modelling Framework

3.1 Model Training and Evaluation Strategy

All models were trained using 10-fold cross-validation (CV), that is, during the training phase, models were repeatedly trained on 90% of the training subset and validated on the remaining 10%, rotating folds, to find optimal hyperparameters. This cross-validation approach helps prevent overfitting and ensures that performance metrics are reliable. We evaluated the following models:

- Logistic Regression
- Lasso Regression
- Ridge Regression
- Decision Tree
- Random Forest (initial grid-search train_small)
- Gradient Boosting (initial grid-search on train_small)

All models were evaluated with a weighted approach in order to account for the imbalance and additionally with an unweighted approach as well to being able to compare results afterwards and assess for the effects of weighting. Random Forest as well as Gradient Boosting were first trained on the 10% subset of the training data set to balance finetuning and computational feasibility, followed by afterwards training on the full training set with the beforehand found optimized grid search.

Our optimization process was two-staged, with the first stage optimizing for ROC-AUC and with the second, optimizing the decision threshold to minimize the expected costs. AUC measures ranking quality independent of chosen thresholds and as such, acts as a robust metric for general optimization of typical binomial classification problems. We optimized all our models during the training for AUC. Meanwhile, business decisions in the financial services industry (and generally in most industries) are optimized by effective costs, not by sensitivity or AUC alone, and costs are – given asymmetric costs of misclassification as the context of credit card defaults, dependent on the cost function and the corresponding decision threshold that minimizes these costs under the given (best) AUC-models. With this goal of minimizing expected loss, we constructed a typical cost-function and optimized the threshold of the already for AUC optimized models such as minimizing the expected costs. These thresholds were then kept constant per model and used to determine how the trained model would perform on the test data, as well as on the hold-out data set.

In short: In a first step the models were optimized for AUC and their respective threshold that minimizes expected costs. In a second step, these models were evaluated on the test set, and – keeping the threshold constant, the model with the lowest expected costs was chosen as the best model. Consequently, models that achieve lower AUC but enable better cost-sensitive decision thresholds would be chosen instead of alternatives – we thus clearly distinguish between ranking performance (AUC) and decision performance (Expected Costs).

3.2 Handling Class Imbalance

The dataset exhibits substantial class imbalance ($\approx 78\%$ non-defaults vs. $\approx 22\%$ defaults). To correct the class imbalance, default observations were assigned a weight of

$$w_{\text{default}} = \frac{0.78}{0.22} \approx 3.55$$

with non-default observations left unchanged at 1. As a result, errors on default cases received approximately 3.5 times more importance during model training and in this sense, leading to a balanced dataset. We still used a non-weighted approach as well for every model to being able to assess the impact of weights on our evaluation strategy.

3.3 Cost Structure and Threshold Optimization

We constructed a cost function typical in credit risk. For each trained model, predicted probabilities on the **training set** are used to determine the threshold t^* that minimizes expected cost:

$$\text{Cost}(t) = C_{FN} \cdot FN(t) + C_{FP} \cdot FP(t)$$

With

- $FN(t)$ = Number of *false negatives* at threshold t (defaults incorrectly classified as non-defaults)
 $FP(t)$ = Number of *false positives* at threshold t (non-defaults incorrectly classified as defaults)
- C_{FN} = Cost of a false negative (missed default, e.g. credit loss)
- C_{FP} = Cost of a false positive (rejected good customer, opportunity cost)
- $\text{Cost}(t)$ = Total economic loss induced by decision threshold t

Institution-specific profit and loss figures are generally not publicly available. We therefore approximated the typical costs based on standard credit risks concepts and used the full data set to calculate representative averages correspondingly, using the US Dollar as the model's currency and therefore converting the results (Taiwanese Dollar) to USD with the then average exchange rate of 1/32 USD/NTD in 2005 (when the data was obtained).

3.3.1 Missed Default Costs

We approximated the costs of a missed default by using the outstanding balance in the data set right before defaulting (here the Bill from 2005 X16) times the factor of how much an institution typically loses from a default (usually around 60%). This gives us the following function:

$$\text{Loss}_i = \text{EAD}_i \times \text{LGD}$$

With:

- EAD_i (*Exposure at Default*): outstanding credit card balance of customer i , proxied by the most recent statement balance
- LGD (*Loss Given Default*): fixed at **60%**, reflecting the average unrecovered share of exposure due to missing recovery data

The **average cost per default** was then computed as:

$$C_{FN} = \mathbb{E}[\text{EAD} \mid \text{Default}] \times \text{LGD}$$

In our case, the C_{FN} equaled $\approx \$920$ USD, being consistent with typical costs of defaults for region and year.

3.3.2 Opportunity Cost of Rejecting a Non-Defaulting Customer

The opportunity cost of incorrectly rejecting a creditworthy customer is approximated as the foregone interest income over the expected customer lifetime.

$$\text{OpportunityCost}_i = \text{EAD}_i \times r \times T$$

with:

- EAD_i : average transaction volume or balance of non-defaulting customer i
- r : average interest / provision rate per period
- T : expected customer lifetime (number of periods)

The **average cost per false positive** is therefore:

$$C_{FP} = \mathbb{E}[\text{EAD} \mid \text{Non-Default}] \times r \times T$$

We approximated the average transaction volume per year by calculating the sum of all bills paid for all observations where a non-default resulted. The sum of these 6 months of payments was then multiplied by 2, giving us an approximation of a year's payments. We then assumed an average interest rate of 2% (typical for region and year) and used an expected customer lifetime of 5 years (typical for 'healthy customers'), resulting in C_{FP} of $\approx \$210$ USD and a cost ratio of $\approx 4.5 : 1$, being consistent with literature for approx. region and year.

Thresholds are then searched and optimized to minimize the expected costs over a fine grid of $t \in [0.01, 0.99]$. The selected threshold is then fixed and applied to the test set (out-of sample evaluation) and later the hold-out "real" dataset, therefore avoiding test-set leakage and simulating a real world scenario.

4. Results

4.1 Overview

Table 1 shows a summary of the model's characteristics and the most important results. Besides AUC scores, the optimal threshold and the typical metrics with binomial classification problems, we computed the costs for each model. We cost here represent the monetary consequence of a current credit decision rule applied to a population of one million with an observed 6-month default structure – given this population would have the exact same imbalance of defaults of $\approx 22\%$. One million customers as the comparing metric were chosen to reflect the average Taiwanese retail credit card customer base. Following, the general results of linear and nonlinear models will be discussed with subchapters focusing on AUC vs. business costs, threshold behavior, the impact of weights and variable importance.

Table 1: Model Performance Summary

<i>Model</i>	<i>AUC (Test)</i>	<i>Optimal Threshold</i>	<i>Accuracy (Test)</i>	<i>Sensitivity (Test)</i>	<i>Specificity (Test)</i>	<i>Cost per 1 mio Customers (Train, USD)</i>	<i>Cost per 1 mio Customers (Test, USD)</i>
<i>Log Regression</i>	0.748	0.53	0.738	0.653	0.761	111.56 Mn	110.47 Mn
<i>Lasso Regression</i>	0.748	0.53	0.739	0.653	0.763	111.30 Mn	110.20 Mn
<i>Ridge Regression</i>	0.747	0.52	0.736	0.656	0.759	111.77 Mn	110.30 Mn
<i>Decision Tree</i>	0.767	0.43	0.747	0.673	0.768	102.80 Mn	105.36 Mn
<i>Random Forest</i>	0.795	0.38	0.649	0.788	0.609	94.50 Mn	109.34 Mn
<i>Gradient Boosting</i>	0.793	0.45	0.717	0.735	0.711	104.09 Mn	102.50 Mn

4.1.1 Linear Models

With the AUC hovering around 0.748, the results of the linear models indicate that there are at least to some extent linear relationships between the predictors and the predicted variable, consistent with general results in the field of amounts of payments and delays. Interestingly, the baseline model of the Log Regression performs better than the finetuned Lasso and Ridge Models in terms of AUC, while offering comparable costs with the latter two – only lacking behind 27-17 thousand. These results are consistent with the grid search results (see Appendix A1 & A2) with the chosen lambda being the lowest in the grid, indicating that the smaller the Lasso resp. Ridge effects and therefore the closer to the baseline model, the higher the resulting AUC.

Even though the data set contains groups of strong correlations between variables, the regularization of Lasso and Ridge do not or not significantly add prediction value because the dataset contains a high number of observations, meaning that the coefficients inside the baseline model are already very stable. The usual benefit of eliminating (Lasso) resp. weakening (Ridge)

variables don't apply in this case and end up slightly increasing bias (in terms of AUC) instead – consistent with a general trend of highly stable coefficients that only lose predictability with additional regularization.

4.1.2 Nonlinear Models

The linear model's results of an AUC around 0.748 indicate that the data contains relevant linear patterns but also clearly non-linear relationships that linear predictors cannot capture. This is underscored by the nonlinear models (Decision Tree, Random Forest, Gradient Boosting), which achieve noticeably higher AUC values in the 0.77–0.79 range. This improvement aligns with domain knowledge that credit default behavior is characterized by threshold effects, interaction patterns, and non-linear escalation dynamics. Variables like payment delays, credit limits, and historical bill trends are unlikely to follow smooth linear trends but instead to change stepwise. Nonlinear models are more capable of learning these structures and therefore outperform the linear regressions.

The same observation applies to the costs, achieving up to 15% / 10% (train, test data) lower resulting costs compared to the linear models, while showcasing a higher, although not representative variance of outputs, indicating that linear models show higher stability in terms of ideal thresholds or generally a flatter cost function around the initial ideal threshold level, consistent as of a typically linear, additive approach vs. nonlinearity.

Furthermore, a general trend of higher complexity correlating with lower stable costs and higher AUC can be seen, with the GBM performing best, followed by the Random Forest and lastly the Decision Tree. These results show that the Forest can capture more complex interactions than the Decision Tree, especially in ranking defaulters more effectively and therefore championing the Sensitivity ranking, but at the same time suffering of either overfitting or a steeper cost function around the threshold optimum. GBM corrects for these by correcting the mistakes of the previous models, enabling a higher stability of predicted costs what is especially seen with a lower variance of expected costs.

Computationally wise, the linear models and the Decision Tree could all be trained in around 10 seconds, while the Forest and GBM would need up to 400 seconds. While substantially higher, our approach also shows that depending on the specific training set up (small training set first for finding a better grid), training more complex models are still timewise feasible and therefore, time as such takes on less relevance for business cases too.

4.1.3 Business Cost Comparison

Table 2 shows how the different models would affect expected business costs in 2005 Taiwan, with the costs corresponding to the expected savings from one large credit approval round, assuming that defaults are observed over a six-month horizon. Given an average retail credit company with a customer base of one million, the models would generally save costs of up 100 million or 49% compared to if no customer selection would take place. With the Log Regression model acting as the baseline, the savings in costs can increase by up to 7.23% - equivalent to 8 million, if the best model would be chosen.

Table 2: Model Performance and Cost-Optimized Results

<i>Model</i>	<i>AUC (Test)</i>	<i>Optimal Threshold</i>	<i>Cost* (Train, USD)</i>	<i>Cost* (Test, USD)</i>	<i>Cost* Savings vs. no Customer Selection** (Test, USD)</i>	<i>Cost Savings vs. no Customer Selection** in % (Test)</i>	<i>Cost Saving vs. baseline model (Log Regression) in % (Test)</i>
<i>Log Regression</i>	0.748	0.53	111.56 Mn	110.47 Mn	91.72 Mn	45%	0.00%
<i>Lasso Regression</i>	0.748	0.53	111.30 Mn	110.20 Mn	92.97 Mn	45%	0.23%
<i>Ridge Regression</i>	0.747	0.52	111.77 Mn	110.30 Mn	91.88 Mn	45%	0.15%
<i>Decision Tree</i>	0.767	0.43	102.80 Mn	105.36 Mn	96.83 Mn	48%	4.63%
<i>Random Forest</i>	0.795	0.38	94.50 Mn	109.34 Mn	92.84 Mn	46%	1.02%
<i>Gradient Boosting</i>	0.793	0.45	104.09 Mn	102.47 Mn	99.71 Mn	49%	7.23%

* per 1 million customers

** refers to the case where all customers would have been accepted, i.e. no selection took place. The expected costs for this case are estimated at USD 202.18 Mn.

Note: All costs are expressed as expected values at the decision date and should be interpreted as economic consequences of a single credit approval round, not as accumulated losses over time.

4.2 AUC vs. Business Cost

While the Area Under the ROC Curve (AUC) is the standard metric for evaluating imbalanced classification problems, it does not uniquely determine an economically optimal decision rule. The AUC summarizes a model's ranking ability across all possible thresholds, whereas real-world credit decisions require the selection of a single operating point.

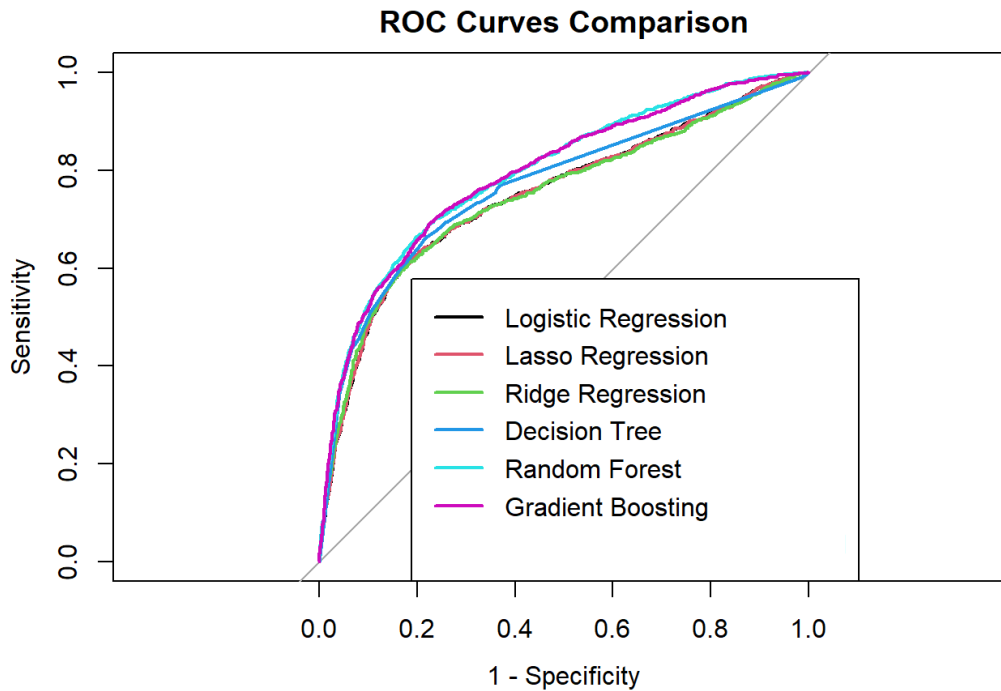


Figure 1: ROC-Curves Comparison all Models, Test set

Figure 1 illustrates that the top-performing models, namely Random Forest and Gradient Boosting, achieve very similar AUC values. However, Figure 2 shows that these models differ substantially in their resulting economic costs once a cost-optimal threshold is applied. This inconsistency highlights that a higher-ranking performance in terms of AUC does not automatically translate into lower business losses.

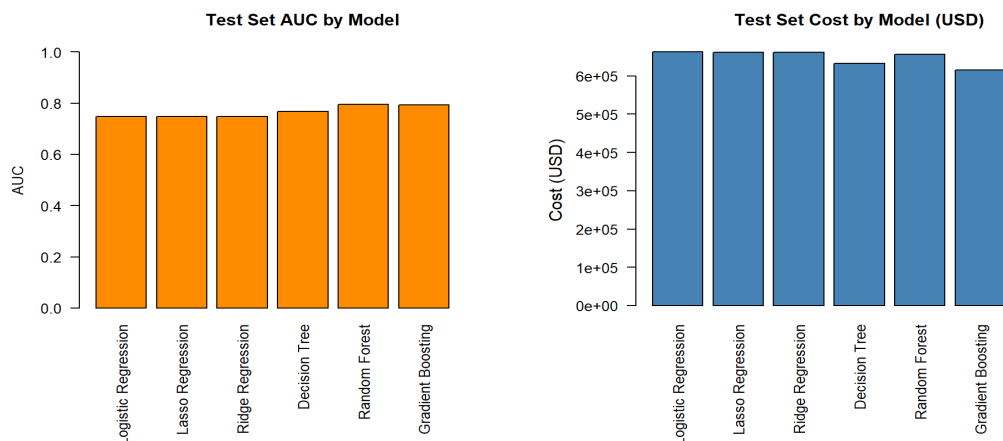


Figure 2: AUC vs. Cost Comparison, Test set

The cost function explicitly penalizes false negatives (missed defaulters) and false positives (rejected profitable customers) with asymmetric economic weights. As a result, the threshold that minimizes expected cost is generally not located at the point of maximal AUC or maximal classification accuracy. It instead reflects the trade-off between optimizing directly for the lowest costs in each scenario and achieving lower costs for this specific case or optimizing for general performance in terms of AUC and afterwards choosing the best threshold, achieving not the

lowest cost for the given scenario overall but optimizing for stability of expected costs and therefore achieving lower costs generally.

This effect is clearly shown by the Random Forest model. Despite achieving the highest AUC among all candidates, it does not yield the lowest out-of-sample costs. In contrast, Gradient Boosting achieves slightly lower AUC values but outperforms Random Forest in terms of realized economic costs on the test data set, while achieving lower variance as well. These findings highlight that models must be evaluated not only by their ranking performance, but also by how effectively they translate predicted risk scores into economic decisions. The implications of this translation, especially with respect to threshold selection and generalization, are discussed in the following section.

4.3 Threshold Behaviour and Generalization

While overall model performance can be summarized by global metrics such as AUC, actual credit decisions are taken at a single operating point defined by a decision threshold. In this context, the stability and generalization of the cost-optimal threshold are of central importance to business decisions.

Figures 3, 4 and 5 show the relationship between decision thresholds and total expected costs for selected models, comparing training and test data. Several clear patterns emerge:

First, **cost-optimal thresholds changes substantially across models**, even when AUC values are similar. Linear models (logistic see figure 3, Lasso, Ridge) exhibit optimal thresholds close to 0.5 (≈ 0.52 – 0.56), reflecting relatively symmetric and well-calibrated probability distributions. Their cost curves are comparatively flat around the optimum, indicating a possible higher robustness to small threshold misspecifications.

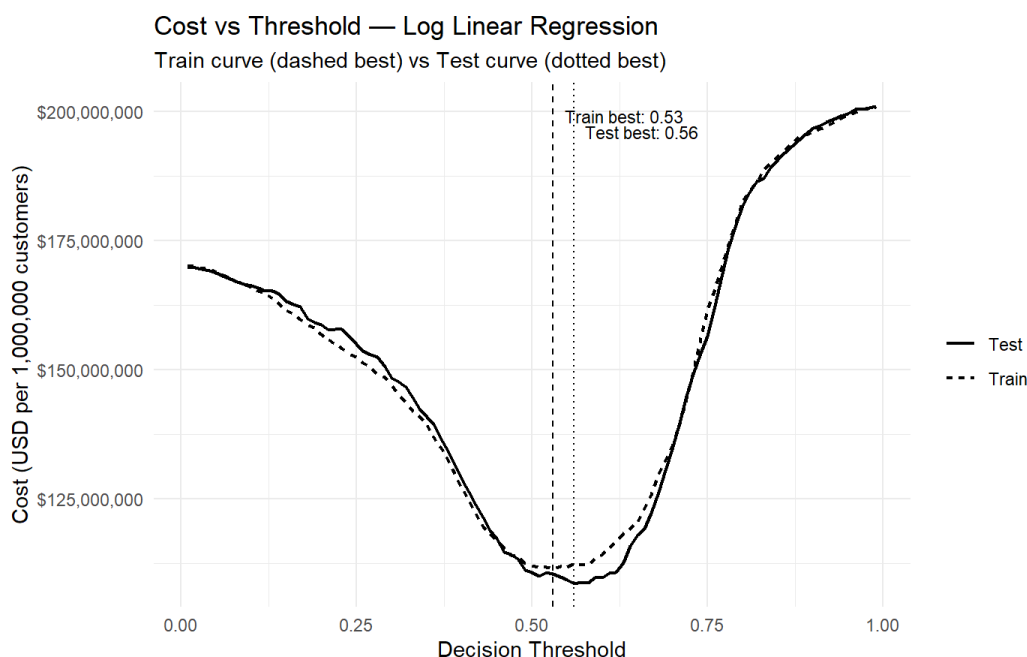


Figure 3: Cost vs Threshold of the Log Regression – Test and Train set

Second, **nonlinear models display significant different threshold behavior**. The Random

Forest (Figure 4) achieves a low training-set optimum around 0.38, but the corresponding test-set optimum shifts upward to approximately 0.50. This widening train–test gap suggests small overfitting at the decision level: while the ranking performance (AUC) remains strong, the mapping from predicted probabilities to economically optimal decisions is less stable out-of-sample.

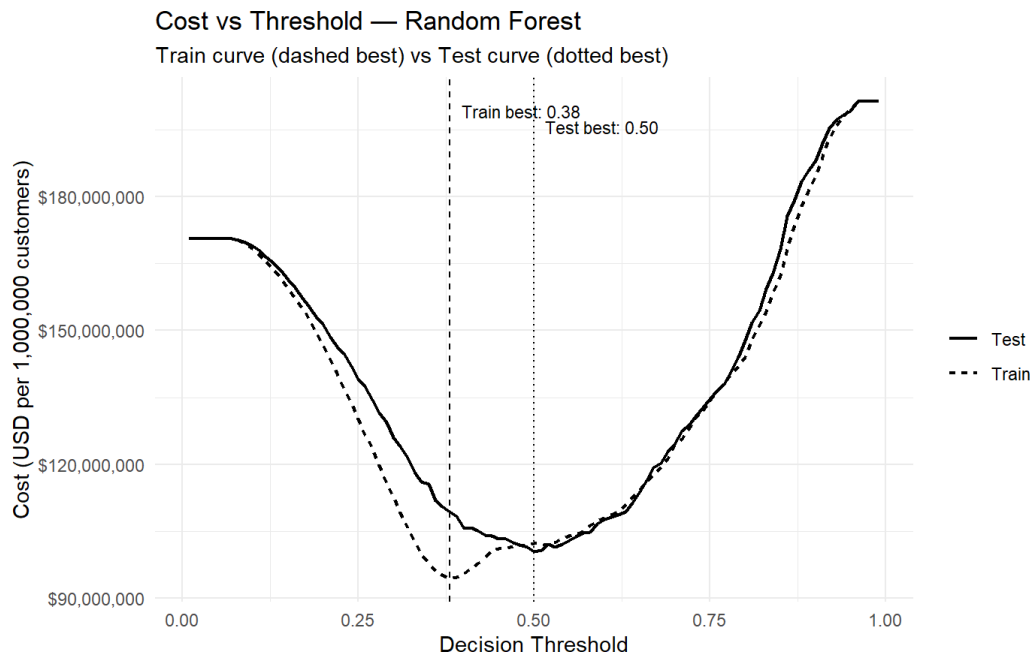


Figure 4: Cost vs Threshold of the Random Forest Model – Test and Train set

In contrast, **Gradient Boosting exhibits superior threshold stability**. Figure 5 shows a narrow alignment between training- and test-optimal thresholds (train $\approx$ 0.45, test $\approx$ 0.51), combined with the lowest observed test-set costs. This indicates not only strong discriminatory power, but also better calibration and generalization of probability estimates in economically relevant scenarios. These findings are especially reflected in the expected costs, with the RF model saving nearly 15% more on the train set compared to the baseline log regression model while only saving 1% more

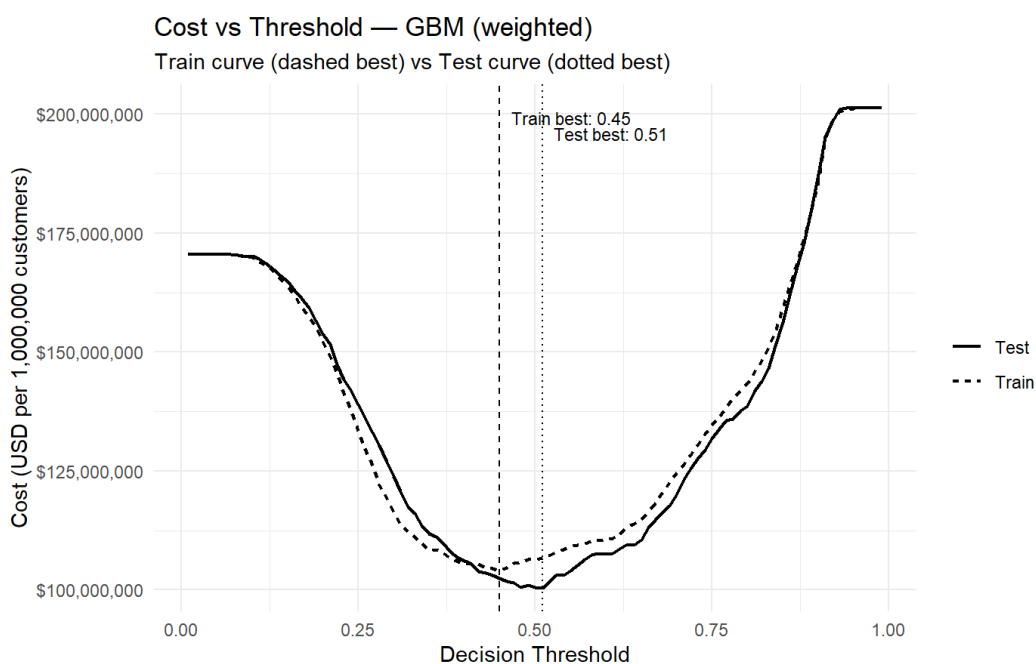


Figure 5: Cost vs Threshold of the Gradient Boosting Model – Test and Train set

on the test set, while the GBM stays constant with savings compared to the baseline at around 6-7% in both train and test set.

The results highlight that stability varies significantly by model class and that models that combine strong ranking in AUC with well-generalizable thresholds offer the highest benefit to businesses from a decision perspective.

4.4 Weighted vs. Unweighted

Our approach of computing predictions for the same models in a weighted and unweighted scenario offers additional information about how modeling specifications influences expected costs of businesses.

Class weighting changes the loss landscape during training by penalizing false negatives more strongly, thereby shifting predicted probabilities upwards for default observations. This improves sensitivity at relevant decision thresholds but may distort the global ranking between classes- in this sense, changing as of who is ranked riskier to whom, leading to unchanged or slightly reduced AUC values, as Table 3 shows. At the same time, the ideal operating point is changed due to the shifted probabilities for default observations and as such the changed global ranking.

Table 3: AUC weighted vs. unweighted

<i>Model</i>	<i>AUC, weighted (Test)</i>	<i>AUC, unweighted (Test)</i>
<i>Log Regression</i>	0.748	0.749
<i>Lasso Regression</i>	0.748	0.749
<i>Ridge Regression</i>	0.747	0.747
<i>Decision Tree</i>	0.767	0.778
<i>Random Forest</i>	0.795	0.793
<i>Gradient Boosting</i>	0.793	0.793

A notable contrast arises when comparing weighted and unweighted Gradient Boosting. The unweighted GBM yields substantially lower optimal thresholds (≈ 0.18), reflecting a scoring distribution that requires aggressive classification of high-risk customers to mitigate costly false negatives. While this approach improves recall, it leads to higher variance and inferior cost performance out-of-sample compared to the weighted model. Table 4 especially shows that an initial lower expected cost level on the train set of the unweighted model must not translate into better test results as well.

Table 4: Expected Costs of GBM weighted vs. unweighted

<i>Model</i>	<i>AUC (Test)</i>	<i>Optimal Threshold</i>	<i>Cost per 1 mio Customers (Train, USD)</i>	<i>Cost per 1 mio Customers (Test, USD)</i>	<i>Cost Savings vs. no Customers Selection - per 1 mio Customers (Test, USD)</i>
<i>GBM, weighted</i>	0.793	0.45	104.09 Mn	102.47 Mn	99.71 Mn
<i>GBM, unweighted</i>	0.795	0.18	97.44 Mn	103.73 Mn	98.45 Mn

Figures 6 and 7 show the cost vs. threshold curves for the GBM model with the weighted and unweighted approach. The Figures underscore the finding of a changed global ranking, resulting in different cost functions given thresholds. The cost curve of the weighted model is much flatter around the optimum compared to the unweighted, indicating higher robustness to threshold misspecifications and in this sense a higher generalized benefit to businesses.

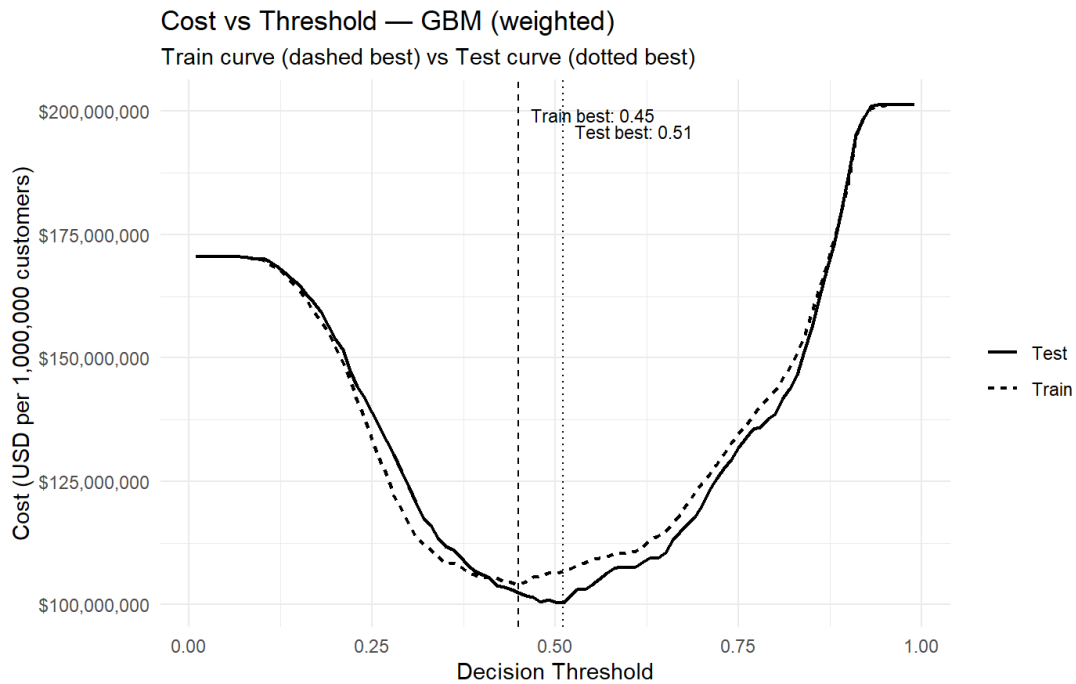


Figure 6: Cost vs Threshold of the Gradient Boosting Model weighted – Test and Train set

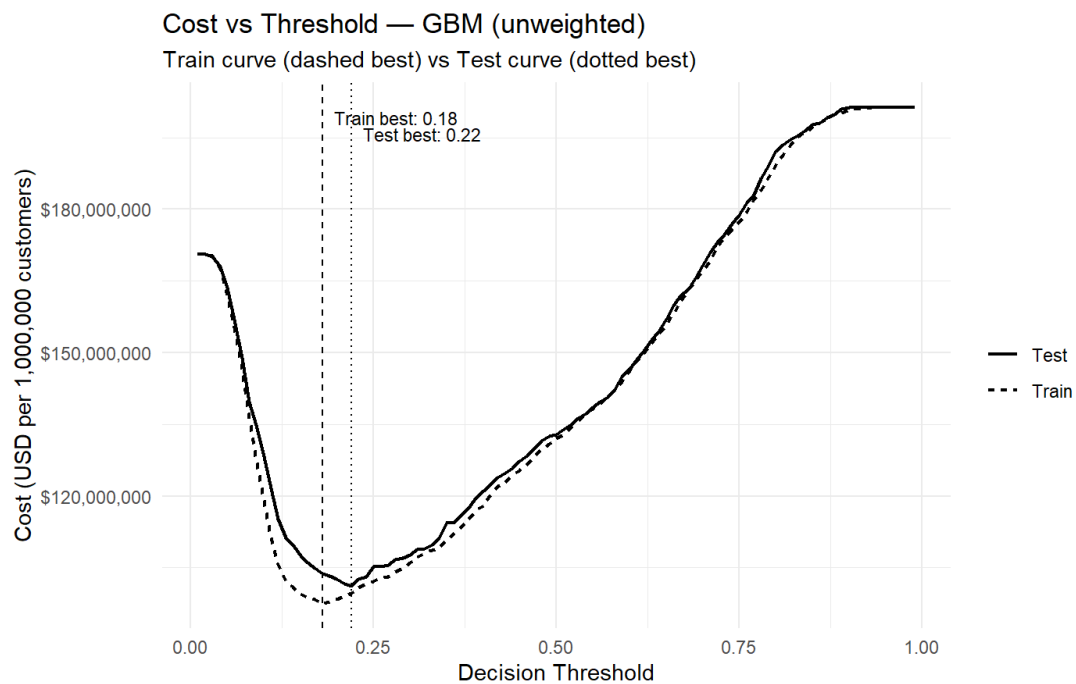


Figure 7: Cost vs Threshold of the Gradient Boosting Model unweighted – Test and Train set

4.5 Variable Importance

Across all models, recent payment behavior is revealed as the strongest driver of default risk (See Appendix A). Variables such as **MaxDelay** and **PAY_SEPT** consistently rank as the most important predictors, especially in nonlinear models. This reflects the intuitive fact that recent delinquency strongly signals future default.

Payment amounts, total payments, and credit limits contribute additional information, while **demographic variables (age, education, marital status, gender) show very low importance** in all models except for the Ridge and Lasso approaches, confirming that the additional regularization has indeed not added value. Furthermore, variables such as payment delays, credit limits and historical bill trends are unlikely to be represented smoothly by probability outcomes, but instead likely follow stepwise escalation patterns when assessing their significance to defaults, showing why simple linear models, not able to represent such patterns accurately, are outperformed by nonlinear models.

Comparing weighted and unweighted Gradient Boosting reveals that weighting does not substantially change the ranking of variables (except for a swap on top of the ranking of most important variables). Instead, weighting mainly affects the decision threshold and cost outcomes, rather than the underlying risk patterns learned by the model.

Overall, the findings align well with economic intuition and existing credit-risk literature.

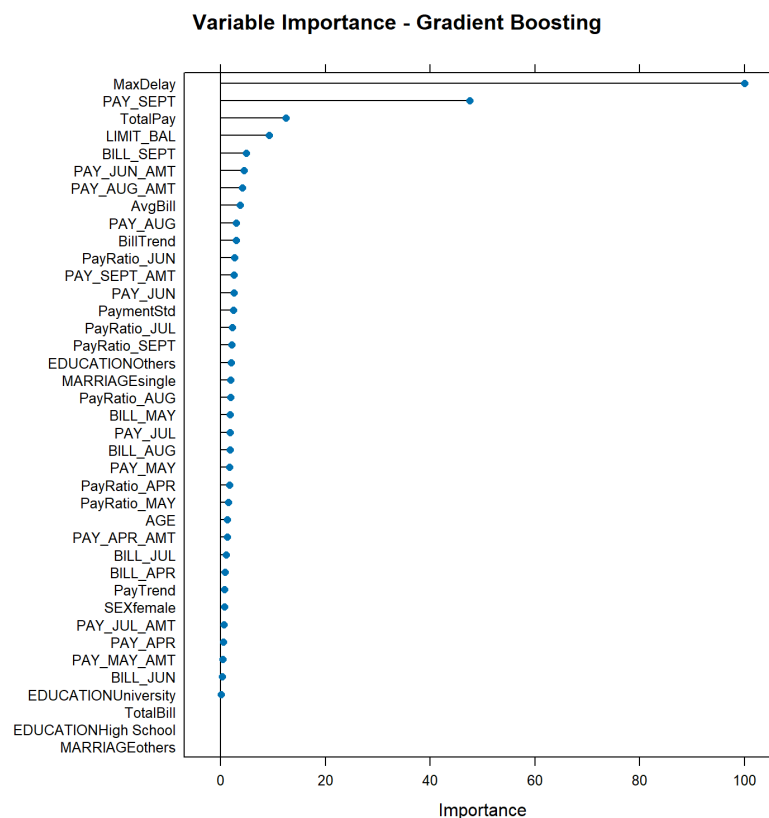


Figure 8: Variable Importance of GBM Model weighted

5. Model Selection and Deployment Simulation

As discussed, performance was assessed primarily by comparing expected costs on the test set, reflecting the operational goal of minimizing financial losses rather than maximizing ranking performance. AUC scores are reported for completeness, but were not used as the primary decision criterion, as they do not incorporate asymmetric misclassification costs.

Based on the results, **Gradient Boosting (weighted)** is selected as the final model with:

- **Optimal threshold:** ≈ 0.45
- **Lowest expected cost on test set**
- **Comparatively stable generalization behavior between train and test data**

We simulated real-world deployment by applying this model to the previously unseen **hold-out “real” data set** results in expected costs of USD 112.67 Million, being inconsistent with the prior stability of results of our best model. We additionally simulated the unweighted model with the same data set, resulting in a similar number (see Table 5). The results show that threshold stability of only train and test set approaches do not yield necessary similar numbers for future predictions, and therefore in the ideal case, should be tested further until the robustness of the optimal threshold would show statistical significance. Important here is to strictly differentiate between such an approach of first stage optimizing for AUC and second stage for threshold and its robustness vs. a direct first stage optimization for threshold, since the second approach would not raise the robustness of the optimal threshold point for new data but contrarily lead to overfitting and less robustness, especially in scenarios of high imbalance, steep cost curves and complex models.

Nevertheless, the deployment exercise illustrates the practical relevance of cost-sensitive evaluation. Even when ranking performance remains stable, realized economic outcomes can vary substantially, underscoring the importance of combining probabilistic models with explicit decision rules.

Table 5: Expected Costs of Deployment Simulation in Comparison

<i>Model</i>	<i>AUC (Test)</i>	<i>Optimal Threshold</i>	<i>Cost per 1 mio Customers (Train, USD)</i>	<i>Cost per 1 mio Customers (Test, USD)</i>	<i>Cost per 1 mio Customers (Simulation, USD)</i>
<i>GBM, weighted</i>	0.793	0.45	104.09 Mn	102.47 Mn	112.67 Mn
<i>GBM, unweighted</i>	0.795	0.18	97.44 Mn	103.73 Mn	112.77 Mn

6. Discussion

This study highlights a central insight often overlooked in applied machine learning: **Maximizing AUC does not guarantee optimal business decisions.**

While AUC is valuable for model comparison, it ignores asymmetric costs and decision thresholds. Weighting schemes and threshold optimization reshape the probability distribution, often reducing AUC slightly while improving economic outcomes and generalizability.

In our case, Random Forest achieves marginally higher AUC, but Gradient Boosting (weighted) consistently minimizes expected losses. This demonstrates why cost-aware evaluation is essential in credit risk modeling.

7. Conclusion

We demonstrate that combining:

- class weighting,
- cross-validation,
- AUC-optimization,
- followed by an explicit cost-based threshold optimization

results in better business decisions than AUC-driven approaches alone within an imbalanced binomial classification problem with an implied high asymmetric payoff structure. Gradient Boosting with cost-minimizing threshold selection results as the economically optimal solution, even when its AUC is not maximal. Furthermore, optimizing the threshold selection for robustness as well would further enhance the benefit for real-world applications.

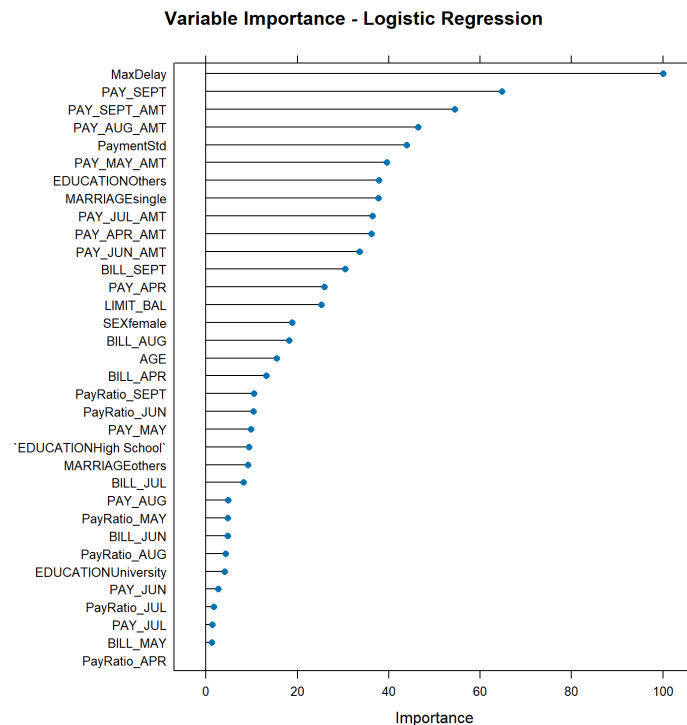
Appendix A: Model Specifications and Hyperparameter Tuning

This appendix summarizes the modeling choices and tuning strategy. All models are trained using 10-fold cross-validation on the training set. Performance is evaluated on a held-out test set, and thresholds are optimized exclusively on training data to avoid information leakage.

A.1 Logistic Regression (Weighted)

- **Model:** Generalized Linear Model (Binomial Logit)
- **Class weights:** Defaults weighted by imbalance ratio (≈ 3.5)
- **Tuning:** No hyperparameters
- **Strengths:** Interpretability, baseline benchmark
- **Limitations:** Linear decision boundary

Result: Competitive AUC but relatively high expected cost due to limited flexibility in capturing nonlinear risk patterns.

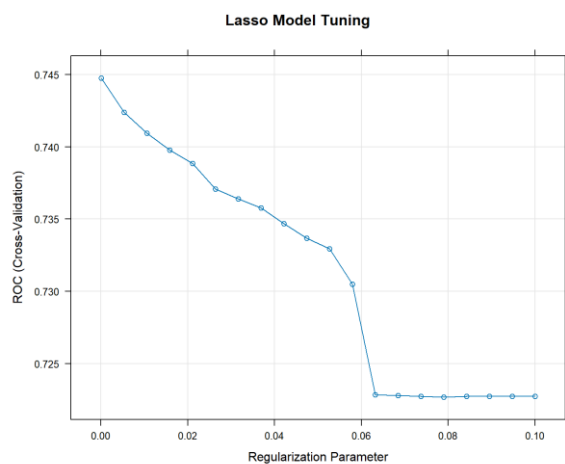


Graphic: Variable Importance of Logistic Regression

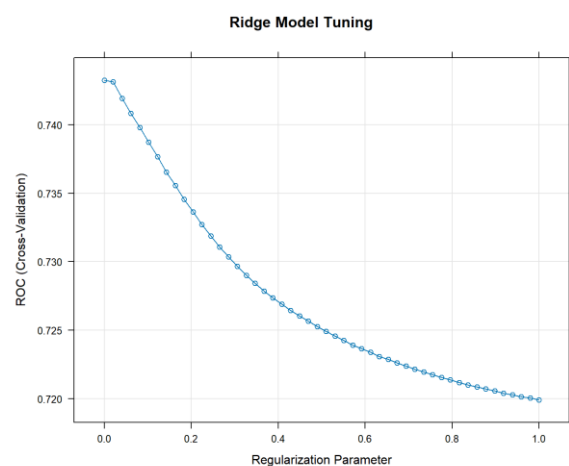
A.2 Lasso and Ridge Regression (Weighted)

- **Model:** Penalized logistic regression (glmnet)
- **Penalty grids:**
 - Lasso ($\alpha = 1$): $\lambda \in [0.0001, 0.1]$, length = 20
 - Best λ : 0.0001
 - Ridge ($\alpha = 0$): $\lambda \in [0.00001, 1]$, length = 50
 - Best λ : 0.00001
- **Data scaling:** Required (z-score)
- **Class weights:** Same as Logistic Regression

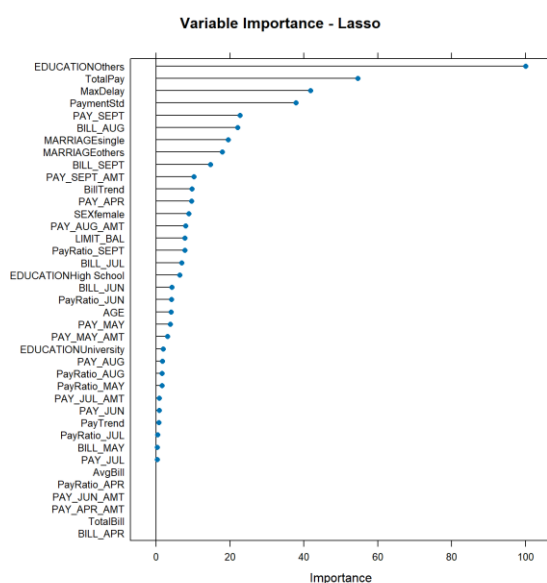
Result: Slightly lower AUC but slightly lower costs



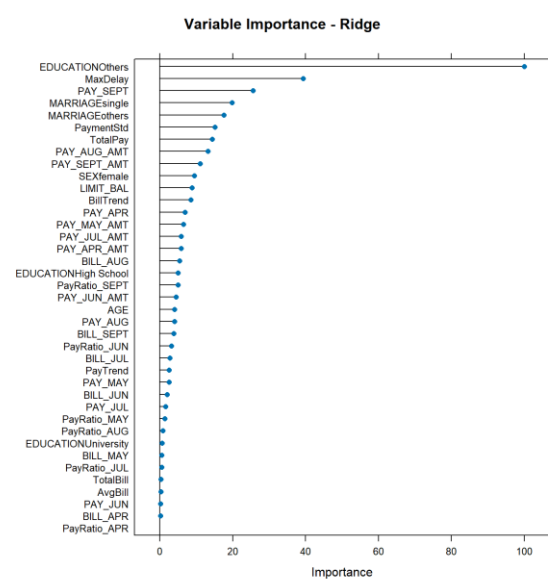
Graphic: Tuning-Grid of Lasso Model



Graphic: Tuning-Grid of Ridge Model



Graphic: Variable Importance of Lasso Model

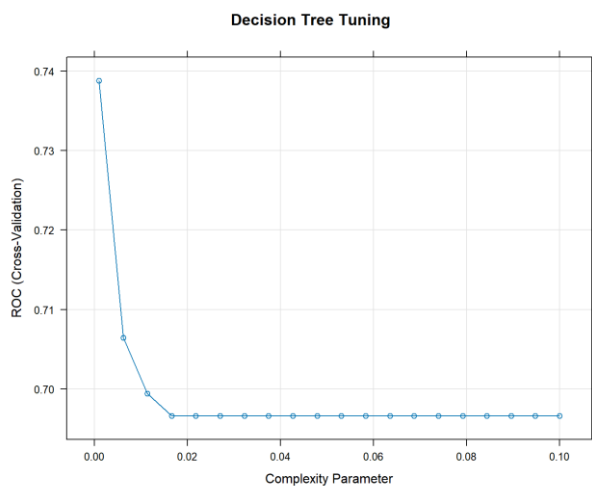


Graphic: Variable Importance of Ridge Model

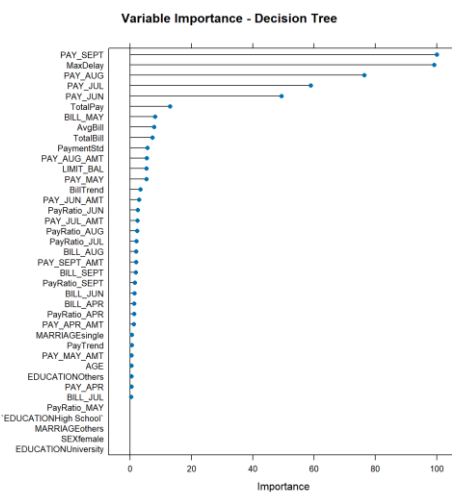
A.3 Decision Tree (Weighted)

- **Model:** CART (rpart)
- **Tuning parameter:**
 - Complexity parameter: $cp \in [0.001, 0.1]$, length = 20
 - Best cp : 0.001
- **Class weights:** Applied via case weights
- **Interpretability:** High

Result: Performs competitively in cost terms, but prone to variance and less stable thresholds than ensemble methods.

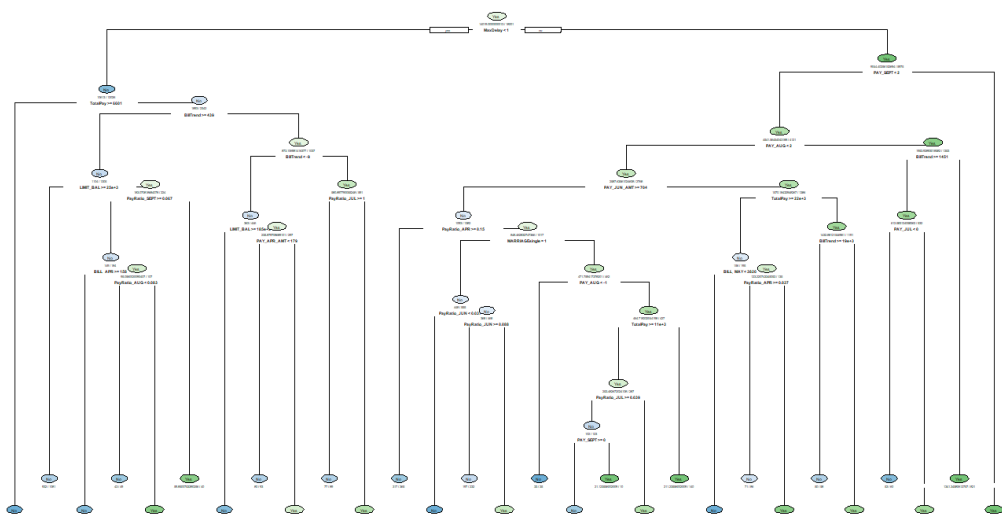


Graphic: Tuning-Grid of Tree Model



Graphic: Variable Importance of Tree Model

Decision Tree Structure

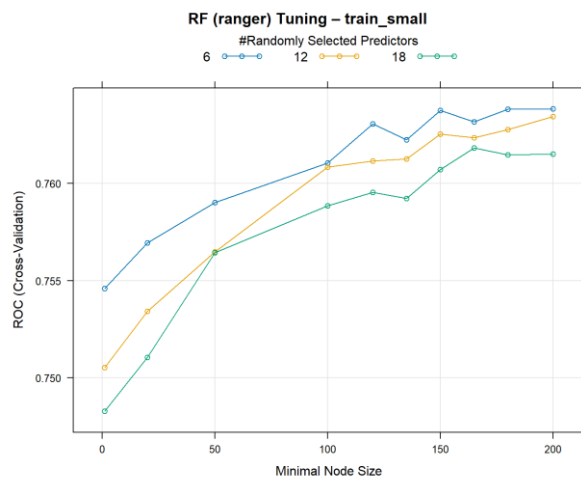


Graphic: Decision Tree Structure of Tree Model

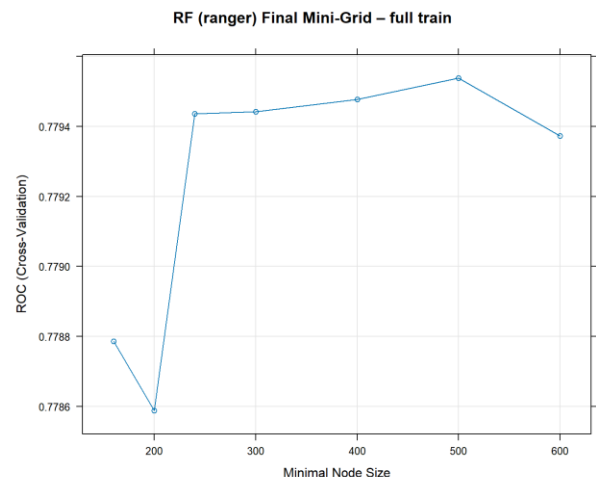
A.4 Random Forest (Weighted)

- **Model:** ranger
- **Tuning strategy:**
 - Initial grid search on 10% stratified training subset (train_small)
 - Parameters:
 - $mtry \in \{\sqrt{p}, p/3, p/2\}$
 - Ideal $mtry = 6$
 - $min.node.size \in \{1, 20, \dots, 200\}$
- **Final fit:** Refined grid around optimal node size on full training data
 - Grid of 0.8-3.0 best-node size of train_small result
 - Ideal node-size = 500
- **Number of trees:** 500
- **Class weights:** Applied

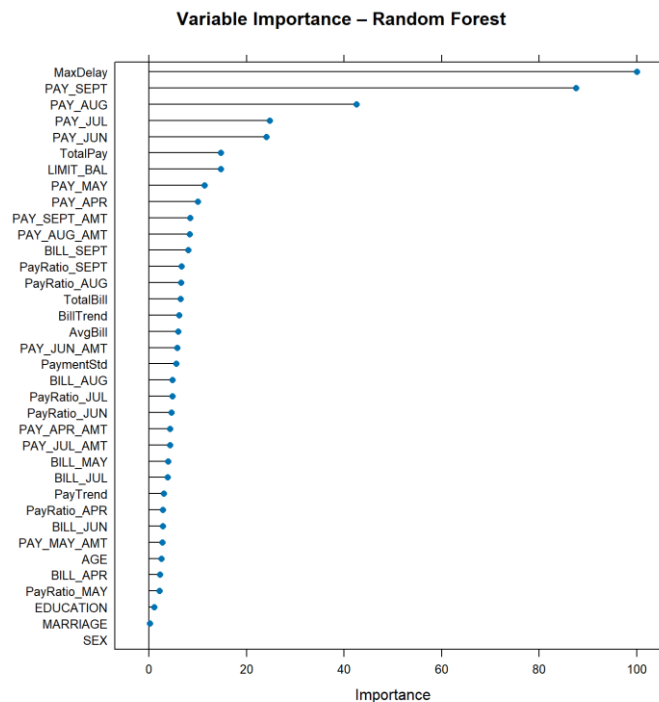
Result: Highest AUC among all models, but larger divergence between training and test costs, indicating mild overfitting in cost space respectively less robust to changing ideal thresholds



Graphic: Tuning-Grid of Random-Forest Model on train_small



Graphic: Tuning-Grid of Random-Forest Model on train_full – the full training set



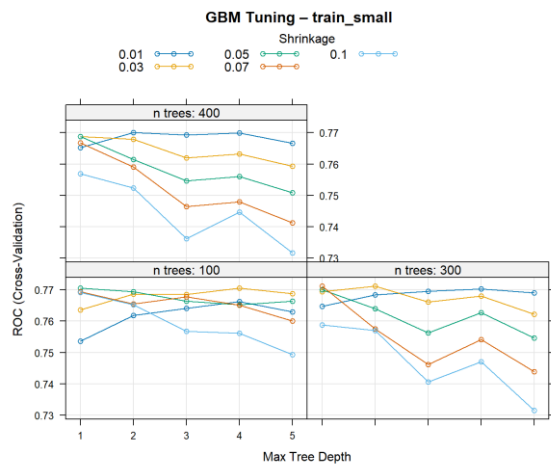
Graphic: Variable Importance of RF Model

A.5 Gradient Boosting (Weighted & Unweighted)

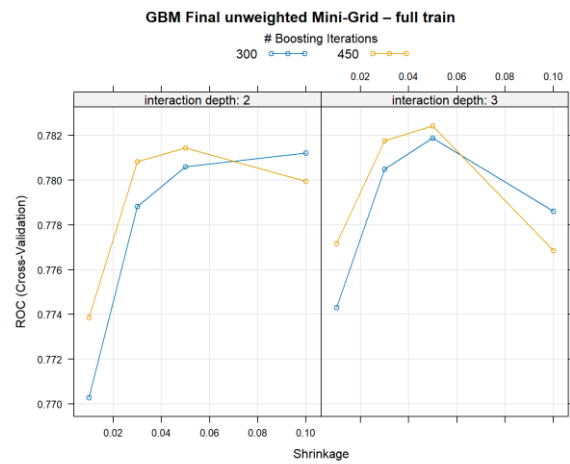
- **Model:** gbm
- **Initial tuning on train_small:**
 - n.trees $\in \{100, 300, 400\}$
 - interaction.depth $\in \{1, 2, 3, 4, 5\}$
 - shrinkage $\in \{0.01, 0.03, 0.05, 0.07, 0.1\}$
 - minimum obs in terminal nodes = 10
- **Final grid:**
 - n.trees $\in \{\text{best}, 1.5 \times \text{best}\}$
 - interaction.depth, shrinkage fixed at optimal values
 - Weighted: depth = 2, shrinkage = 0.03, trees = 300
 - Unweighted: depth = 3, shrinkage = 0.05, n.trees = 450
- **Variants:**
 - Unweighted
 - Weighted using class imbalance ratio

Result: The **weighted Gradient Boosting model** achieves the lowest expected cost on the test set and shows stable threshold behavior between training and test data. The unweighted achieves

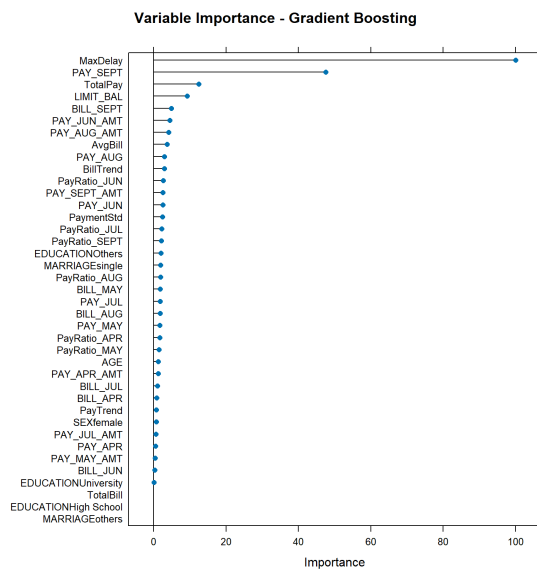
lower AUC, lower lower expected cost on train data set, but higher costs on test data set, indicating less robustness due to imbalance



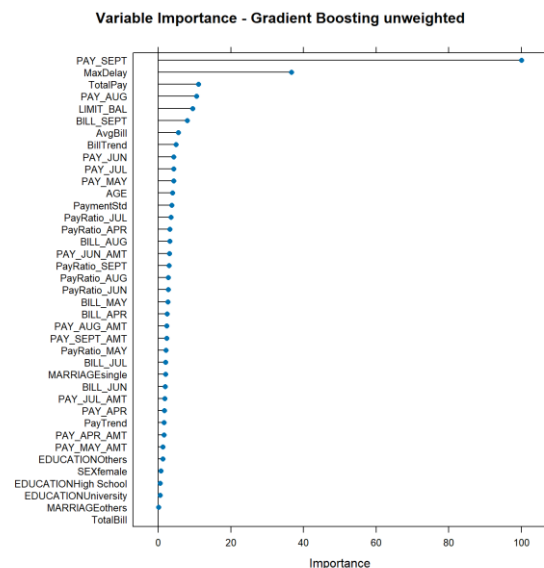
Graphic: Tune-Grid of GMB Model weighted



Graphic: Tune-Grid of GMB Model unweighted



Graphic: Variable Importance of GBM Model weighted



Graphic: Variable Importance of GBM Model unweighted